Week 2 Day-by-Day Plan: FastAPI Service + Batch Processing

Building the REST API Around Your Week 1 ML Pipeline

> **Goal**: By end of Week 2, you have a fully working API that accepts text via HTTP, runs toxicity classification, and returns structured JSON responses — testable via Swagger docs, curl, or any HTTP client.

What You Already Have (from Week 1)

Your skeleton is 70% built. Here's the inventory:

File	Status	What It Does
`app/main.py`	✅ Done	FastAPI app, CORS, startup event
`app/config.py`	⚠️ Has a bug	Settings, thresholds — missing `categories` field
`app/models/moderator.py`	✅ Done	`ContentModerator` class with `moderate()` and `moderate_batch()`
`app/schemas/moderation.py`	✅ Done	All Pydantic request/response models
`app/utils/language.py`	✅ Done	Language detection with alias mapping
`app/api/routes.py`	🔴 Needs work	`POST /moderate` and `POST /moderate/batch` return 501
`tests/test_moderator.py`	🔴 Placeholder	All tests are `assert True`

This week you're implementing the 🔴 items and fixing the ⚠️ bug.

Pre-Setup (10 min)

```powershell
```

```
cd d:\AI_ML_Projects\multilingual_content_moderator
.\venv\Scripts\Activate

# Verify FastAPI is installed
python -c "import fastapi; print(f'FastAPI {fastapi.__version__}')"
python -c "import uvicorn; print('Uvicorn OK')"
python -c "import pydantic; print(f'Pydantic {pydantic.__version__}')"

# If any fail:
pip install fastapi uvicorn pydantic pydantic-settings
```

Day 1 (Monday) — FastAPI Fundamentals + Fix Existing Bugs (2-3 hours)

What to learn

Before writing API code, you need to understand **how FastAPI works**. These are the core concepts:

1. **What is FastAPI?** — A Python web framework for building APIs. It auto-generates interactive docs (Swagger UI) and validates all inputs/outputs using Pydantic.
2. **What is an endpoint?** — A URL + HTTP method that does something. `POST /api/v1/moderate` = "send text to this URL and get toxicity scores back"
3. **What is Pydantic?** — A data validation library. Your `ModerationRequest` and `ModerationResult` classes define exactly what the API accepts and returns. If someone sends invalid data, FastAPI auto-rejects it with a clear error.
4. **What is uvicorn?** — The server that runs your FastAPI app. Like how a waiter (uvicorn) takes orders to the kitchen (your FastAPI code).

What to do

Step 1: Run the existing app and explore Swagger docs

```powershell
cd d:\AI_ML_Projects\multilingual_content_moderator
.\venv\Scripts\Activate
uvicorn app.main:app --reload --port 8000
```

Then open `http://localhost:8000/docs` in your browser. You'll see all endpoints listed. Try clicking on `GET /api/v1/health` → "Try it out" → "Execute". You should get a response. Try `GET /api/v1/languages` — it should return your supported languages. Try `POST /api/v1/moderate` — it will return 501 (that's what we'll fix on Day 2).

Step 2: Fix the `categories` bug in config.py `routes.py` line 74 references `settings.categories` but it doesn't exist in `config.py`. Add the `categories` field to your `Settings` class:

```python
# Add this to your Settings class in config.py:
categories: list = ["toxic", "non-toxic"]
```

Note: The CitizenLab model only outputs "toxic" and "non-toxic" (2 labels), unlike the Unitary model which had 6 categories. Your config should match your actual model.

Step 3: Fix the `GET /categories` endpoint After adding the field, restart the server and test `GET /api/v1/categories`. It should now return `{"categories": ["toxic", "non-toxic"]}`.

Step 4: Understand the request/response flow Read through `app/schemas/moderation.py` carefully. Trace the data flow:

```
Client sends JSON → FastAPI validates against ModerationRequest
    → Your code processes it
    → You build a ModerationResult
    → FastAPI validates and returns JSON
```

Done when

- [ ] Server runs on `http://localhost:8000`
- [ ] Swagger docs load at `http://localhost:8000/docs`
- [ ] `GET /health` returns model info
- [ ] `GET /languages` returns language list
- [ ] `GET /categories` returns category list (bug fixed)
- [ ] You understand the request → processing → response flow
- [ ] `git commit -m "Day 1: FastAPI running, categories bug fixed"`

Day 2 (Tuesday) — Implement POST /moderate Endpoint (2-3 hours)

What to learn

This is the **core endpoint** of your entire project. A client sends text, you classify it, and return structured results. Concepts to understand:

1. **Request body** — The JSON the client sends. Validated by `ModerationRequest`.
2. **Dependency injection** — How `get_moderator()` loads the model once and reuses it.
3. **Response model** — `ModerationResult` defines exactly what gets returned.
4. **Error handling** — What happens when input is invalid or the model fails.

What to do

Step 1: Implement the `/moderate` endpoint in `routes.py` Replace the TODO block with actual logic. The flow is:

1. Get the moderator (model loaded once, reused)
2. Call `moderator.moderate(text)` — this handles language detection internally
3. Transform the result dict into a `ModerationResult` Pydantic model
4. Return it

Step 2: Handle the schema mismatch Your `moderator.moderate()` returns a dict with `categories` as:

```python
{"toxic": {"score": 0.85, "flagged": True}}
```

But your Pydantic `ModerationResult` expects `categories` as:

```python
{"toxic": CategoryResult(score=0.85, flagged=True)}
```

You need to convert the plain dicts to `CategoryResult` objects.

Step 3: Add input validation and error handling

- What if the text is empty? (Pydantic handles this — `min_length=1`)
- What if the model throws an error? (Wrap in try/except)
- What if text is too long? (Pydantic handles this — `max_length=5000`)

Step 4: Test via Swagger Go to `http://localhost:8000/docs`, click `POST /api/v1/moderate`, and test with:

- English toxic: `"You are a disgusting person"`
- English clean: `"Have a wonderful day!"`
- Hindi toxic: `"तुम बेवकूफ हो"`
- Arabic clean: `"شكراً لك على مساعدتك"`

Done when

- [ ] `POST /moderate` returns proper JSON with verdict, categories, confidence
- [ ] Tested with English, Hindi, and Arabic texts
- [ ] Error handling works for invalid inputs

- [ ] Language is auto-detected and returned in response
- [ ] `git commit -m "Day 2: POST /moderate endpoint implemented"`

Day 3 (Wednesday) — Implement POST /moderate/batch Endpoint (2-3 hours)

What to learn

1. **Batch processing** — Processing multiple texts in one API call (more efficient than calling /moderate 100 times)
2. **Aggregate statistics** — Returning summary info (total flagged, total clean)
3. **Performance tracking** — Timing the entire batch and reporting it

What to do

Step 1: Implement the `/moderate/batch` endpoint The flow:

1. Receive a list of texts (up to 100)
2. Loop through each, calling `moderator.moderate()` for each one
3. Collect all individual `ModerationResult` objects
4. Count flagged vs clean
5. Track total processing time
6. Return `BatchModerationResult` with everything

Step 2: Add batch-specific validation

- Empty list? Reject.
- More than 100 texts? Reject (already handled by Pydantic `max_length=100`).
- Individual text too long? Each text still validated.

Step 3: Test with mixed-language batches Send a batch of 10+ texts mixing English, Hindi, and Arabic, toxic and clean. Verify the counts and individual results are correct.

Done when

- [ ] `POST /moderate/batch` processes multiple texts
- [ ] Returns individual results plus aggregate counts
- [ ] Processing time tracked and returned
- [ ] Tested with 10+ mixed texts
- [ ] `git commit -m "Day 3: Batch endpoint implemented"`

Day 4 (Thursday) — Input Validation, Error Handling & Edge Cases (2-3 hours)

What to learn

1. **Input sanitisation** — Cleaning inputs before processing
2. **Custom error responses** — Returning useful error messages
3. **Edge cases from Week 1** — Short text, empty text, special characters

What to do

Step 1: Add input sanitisation Remember your `classify_safe()` guardrails from Week 1? Port them into the API:

- Minimum text length warning (texts under 3 words)
- Text normalisation (lowercase, reduce repeated chars)
- Handle texts that are just whitespace or special characters

Step 2: Add custom error responses Create proper error responses for:

- Invalid threshold values (already handled by Pydantic ge=0.0, le=1.0)
- Model loading failures
- Unexpected errors during inference

Step 3: Test edge cases through the API Test these through Swagger:

- Very short text: `"hi"`
- Just whitespace: `"  "`
- Just emojis: `"🔥🔥🔥"`
- Very long text (paste a long paragraph)
- Mixed language: `"You are such a bewakoof"`
- Threshold override: Send with `{"text": "...", "threshold": 0.3}`

Done when

- [ ] Short text handled gracefully (returns result with a note)
- [ ] Whitespace-only text rejected or handled
- [ ] Custom threshold works when passed in request
- [ ] Errors return clean JSON (not HTML stack traces)
- [ ] `git commit -m "Day 4: Input validation, error handling, edge cases"`

Day 5 (Friday) — Write Tests (2-3 hours)

What to learn

1. **pytest** — Python's most popular testing framework
2. **TestClient** — FastAPI's built-in way to test endpoints without running a server
3. **Unit vs integration tests** — Testing individual functions vs full API flow

What to do

Step 1: Write unit tests for `ContentModerator` Test the model wrapper directly (no API involved):

- `test_moderate_english_toxic` — toxic text returns `verdict: "toxic"`
- `test_moderate_english_clean` — clean text returns `verdict: "clean"`
- `test_moderate_hindi` — Hindi text detected correctly
- `test_moderate_arabic` — Arabic text detected correctly
- `test_threshold_affects_verdict` — same text, different thresholds, different verdict

Step 2: Write unit tests for language detection

- `test_detect_english` — English text → "en"
- `test_detect_hindi` — Hindi text → "hi"
- `test_detect_arabic` — Arabic text → "ar"
- `test_alias_mapping` — Marathi text → mapped to "hi"

Step 3: Write API integration tests Use FastAPI's `TestClient` to test the full endpoint:

- `test_moderate_endpoint` — POST valid text, check response structure
- `test_batch_endpoint` — POST multiple texts, check counts
- `test_invalid_input` — POST empty text, expect 422
- `test_health_endpoint` — GET /health, check model loaded

Step 4: Run all tests

```powershell
pytest tests/ -v
```

Done when

- [ ] At least 10 tests written
- [ ] All tests pass with `pytest tests/ -v`

- [ ] Tests cover: model, language detection, and API endpoints
- [ ] `git commit -m "Day 5: Tests written — model, language, API"`

Day 6 (Saturday) — End-to-End Testing + Performance (2-3 hours)

What to do

Step 1: Test with curl / Postman Move beyond Swagger — test from the command line:

```powershell
# Single text
curl -X POST http://localhost:8000/api/v1/moderate ^
  -H "Content-Type: application/json" ^
  -d "{\"text\": \"You are a terrible person\", \"threshold\": 0.5}"

# Batch
curl -X POST http://localhost:8000/api/v1/moderate/batch ^
  -H "Content-Type: application/json" ^
  -d "{\"texts\": [\"Hello!\", \"You are stupid\", \"शुक्रिया\"]}"
```

Step 2: Performance benchmarking

- Time single requests (should be <500ms on CPU)
- Time batch of 20 texts
- Time batch of 50 texts
- Note if performance degrades with batch size

Step 3: Test the full flow Run all your Week 1 benchmark texts through the API and compare results with your notebook results. They should match exactly.

Done when

- [ ] curl commands work correctly
- [ ] Performance documented (single text latency, batch throughput)
- [ ] API results match notebook results
- [ ] `git commit -m "Day 6: End-to-end testing, performance benchmarks"`

Day 7 (Sunday) — Documentation + Week 2 Wrap (1-2 hours)

What to do

1. Update `README.md` with API endpoint documentation

2. Add example request/response JSON blocks
3. Add "How to run" instructions for the API
4. Write a LinkedIn post about what you built this week

LinkedIn post template

```
Week 2: Built a REST API around my multilingual toxicity classifier! 🛡️

What I built:
- POST /moderate — classify single text with auto language detection
- POST /moderate/batch — process up to 100 texts in one call
- Auto-generated Swagger docs at /docs
- Per-language threshold calibration (EN: 0.5, HI: 0.15, AR: 0.10)
- Full test suite with pytest

Key learning: The hardest part wasn't the ML model — it was designing
clean API contracts that handle edge cases gracefully.

Next week: Docker containerisation + Streamlit dashboard.

#FastAPI #NLP #Python #MachineLearning #BuildInPublic
```

Done when

- [ ] README updated with API docs
- [ ] LinkedIn post published
- [ ] `git add . && git commit -m "Day 7: Week 2 complete" && git push`

Week 2 Summary

Day	Focus	Hours	Key Output
1	FastAPI fundamentals, fix bugs, run server	2-3h	Server running, Swagger working
2	Implement POST /moderate	2-3h	Core endpoint working
3	Implement POST /moderate/batch	2-3h	Batch processing working
4	Input validation, error handling, edge cases	2-3h	Production-ready endpoints
5	Write tests (model, language, API)	2-3h	10+ tests passing
6	End-to-end testing, performance	2-3h	Full flow validated
7	Documentation, LinkedIn	1-2h	README updated, public post

Day	Focus	Hours	Key Output
Total		**14-19h**	**Working REST API with tests**

Key Concepts You'll Learn This Week

Concept	What It Is	Where You'll Use It
REST API	Standard way for apps to communicate over HTTP	Your entire API design
HTTP Methods	GET (read), POST (create/process)	GET /health, POST /moderate
Request Body	JSON data the client sends	ModerationRequest schema
Response Model	JSON structure the API returns	ModerationResult schema
Status Codes	200 (success), 422 (invalid input), 500 (server error)	Error handling
Pydantic	Auto-validates inputs and outputs	All schemas
Dependency Injection	Load model once, reuse across requests	get_moderator()
TestClient	Test API without running a real server	All API tests
CORS	Allow frontend apps to call your API	Already configured
Swagger/OpenAPI	Auto-generated interactive API docs	/docs endpoint

Files You'll Modify This Week

```
app/
├── config.py            ← Fix: add categories field
├── api/
│   └── routes.py        ← Implement: /moderate and /moderate/batch
├── models/
│   └── moderator.py     ← May need minor tweaks for schema compatibility
└── schemas/
    └── moderation.py    ← May need adjustments as you implement
```

```
tests/
└── test_moderator.py    ← Implement: all test cases
```